



International Islamic University Chittagong
Department of Computer Science and Engineering

Project Report

Project Title Full Web Application Penetration Testing

Course Details

Code	CSE-4744
Title	Computer Security Lab

Submitted To

Name	Md. Ziaul Hoque
Designation	Adjunct Lecturer, Department of CSE
Department	Computer Science and Engineering

Submitted By

Member 1:	Sabrina Afroz Nitu
ID	C223241
Section	8BF
Semester	8th

Submission Date

1/8/26

TABLE OF CONTENTS

Contents	Page No.
Abstract	3
Introduction	4
1.1 Objectives	4
1.2 Scope of the Project	4
1.3 Problem Statement	5
1.4 Significance of the Study	5
2.Literature Review	5
3.Methodology	6
3.1 Penetration Testing Methodology	6
3.2 Project Workflow (Flowchart)	7
4.Experimental Setup	8
4.1 Laboratory Environment	8
5.Tools Used	8
5.1 Kali Linux	8
5.2 DVWA (Damn Vulnerable Web Application)	8
5.3 Burp Suite Community Edition	9
5.4 Mozilla Firefox	9
6.About DVWA	9
7.Vulnerability Assessment and Exploitation	10
7.1 SQL Injection (SQLi)	10-13
7.2 Blind SQL Injection	14-16
7.3 Brute Force Attack	17-20
7.4 Command Injection	21-24
7.5 Cross-Site Request Forgery (CSRF)	25-27
7.6 File Inclusion	28-29
7.7 File Upload Vulnerability	30-32
7.8 DOM-Based Cross-Site Scripting (DOM XSS)	33-35
7.9 Reflected Cross-Site Scripting (XSS)	36-37
7.10 Stored Cross-Site Scripting (Stored XSS)	38-39
8: Results and Discussion	40
9: Conclusion	41

ABSTRACT

Web applications are among the most common targets of cyberattacks due to vulnerabilities arising from insecure coding practices and improper security configurations. Penetration testing is an effective approach to identifying these vulnerabilities before attackers can exploit them.

This project presents a comprehensive web application penetration testing assessment using the Damn Vulnerable Web Application (DVWA) in a controlled laboratory environment. The assessment was conducted on Kali Linux with the DVWA security level configured to Low. Various penetration testing techniques were applied to evaluate the security of the target application.

The project covers ten common web application vulnerabilities, including SQL Injection, Blind SQL Injection, Brute Force Attack, Command Injection, Cross-Site Request Forgery (CSRF), File Inclusion, File Upload Vulnerability, DOM-Based Cross-Site Scripting (XSS), Reflected Cross-Site Scripting (XSS), and Stored Cross-Site Scripting (XSS).

For each vulnerability, the testing methodology, exploitation process, impact, and recommended mitigation techniques are discussed. The results demonstrate how improper input validation, weak authentication mechanisms, and insecure configurations can expose web applications to serious security threats.

This project provides practical experience in ethical hacking and emphasizes the importance of secure software development practices for protecting modern web applications.

Keywords

- Web Application Security
- Penetration Testing
- DVWA
- Kali Linux
- SQL Injection
- Cross-Site Scripting (XSS)
- Burp Suite
- Ethical Hacking
- Vulnerability Assessment
- Cyber Security

1. Introduction

Web applications have become an essential part of modern life. People use web applications for online banking, shopping, education, healthcare, and communication. Because these applications store and process sensitive information, they have become common targets for cyber attackers. Weak authentication, poor input validation, insecure file handling, and configuration errors can expose web applications to serious security risks.

Web Application Penetration Testing is a systematic process of evaluating the security of a web application by identifying and safely exploiting vulnerabilities in a controlled environment. The main purpose is to discover security weaknesses before malicious attackers can exploit them.

In this project, Damn Vulnerable Web Application (DVWA) was used as the target application. DVWA is an intentionally vulnerable web application designed for security education and ethical hacking practice. The testing environment was configured on Kali Linux, and the application security level was set to **Low** to demonstrate common web attacks.

The assessment focused on ten common web application vulnerabilities: SQL Injection, Blind SQL Injection, Brute Force, Command Injection, Cross-Site Request Forgery (CSRF), File Inclusion, File Upload, DOM-Based Cross-Site Scripting (XSS), Reflected XSS, and Stored XSS. The project explains the testing procedure, attack results, security impact, and recommended mitigation techniques for each vulnerability.

1.1 Objectives

The main objectives of this project are:

- To understand the fundamentals of web application penetration testing.
- To identify common security vulnerabilities in DVWA.
- To perform ethical penetration testing in a controlled laboratory environment.
- To analyze the impact of different web attacks.
- To recommend appropriate security measures to prevent these attacks.
- To gain practical experience using penetration testing tools and techniques

1.2 Scope of the Project

This project focuses on security testing of DVWA in a local laboratory environment. The assessment includes identifying, exploiting, and analyzing common web application vulnerabilities without affecting any real-world systems.

The scope includes:

- Setting up the penetration testing environment.
- Performing security testing on DVWA.
- Exploiting ten common vulnerabilities.

- Analyzing security risks.
- Providing mitigation techniques.
- Documenting the complete penetration testing process.

The project does not include testing external websites or unauthorized systems. All testing activities were performed only on the intentionally vulnerable DVWA application for educational purposes.

1.3 Problem Statement

Many web applications contain security weaknesses due to insecure coding practices, poor authentication mechanisms, improper input validation, and configuration errors. These vulnerabilities can allow attackers to steal sensitive information, execute malicious commands, upload harmful files, or gain unauthorized access.

Organizations often fail to identify these vulnerabilities during the software development process. As a result, attackers can exploit these weaknesses and compromise the confidentiality, integrity, and availability of information systems.

This project demonstrates how penetration testing can identify such vulnerabilities before they are exploited by malicious users

1.4 Significance of the Study

Web application penetration testing is an important part of modern cybersecurity. This study provides practical knowledge of identifying and analyzing security vulnerabilities in web applications.

The project helps students understand real-world attack techniques in a safe environment and emphasizes the importance of secure software development practices. It also demonstrates how ethical hacking techniques can improve application security by identifying weaknesses before deployment.

The findings of this project can assist developers, students, and security professionals in understanding common vulnerabilities and implementing appropriate countermeasures.

Literature Review

Several researchers and cybersecurity organizations have highlighted the importance of web application security. Modern web applications frequently suffer from vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), insecure file uploads, and command injection. These vulnerabilities can lead to data breaches, unauthorized access, and complete system compromise.

The Open Worldwide Application Security Project (OWASP) regularly publishes the OWASP Top 10, which identifies the most critical web application security risks. These guidelines are widely used by developers and penetration testers to improve application security.

DVWA (Damn Vulnerable Web Application) is one of the most popular educational platforms for learning ethical hacking and web security testing. It provides intentionally vulnerable modules that simulate real-world web application vulnerabilities. Security professionals and students use DVWA to practice penetration testing techniques without affecting production systems.

In this project, DVWA was selected because it provides a controlled environment for understanding common attack techniques while following ethical hacking principles

Methodology

3.1 Penetration Testing Methodology

Penetration testing is a structured process used to evaluate the security of a web application by identifying and exploiting vulnerabilities in a controlled environment. In this project, the testing followed a standard penetration testing methodology to ensure systematic assessment and accurate documentation.

The methodology consisted of the following phases:

Phase 1: Planning and Environment Setup

In this phase, the penetration testing environment was prepared. Kali Linux was used as the attacker machine, and DVWA (Damn Vulnerable Web Application) was configured on a local Apache server with a MySQL database. The security level of DVWA was set to **Low** to demonstrate common vulnerabilities in an educational environment.

Phase 2: Information Gathering

Basic information about the target application was collected, including the target URL, server technology, and available modules. This phase helped in understanding the application's structure before testing.

Phase 3: Vulnerability Assessment

Each DVWA module was examined to identify potential security weaknesses. The assessment focused on input validation, authentication mechanisms, file handling, and server-side processing.

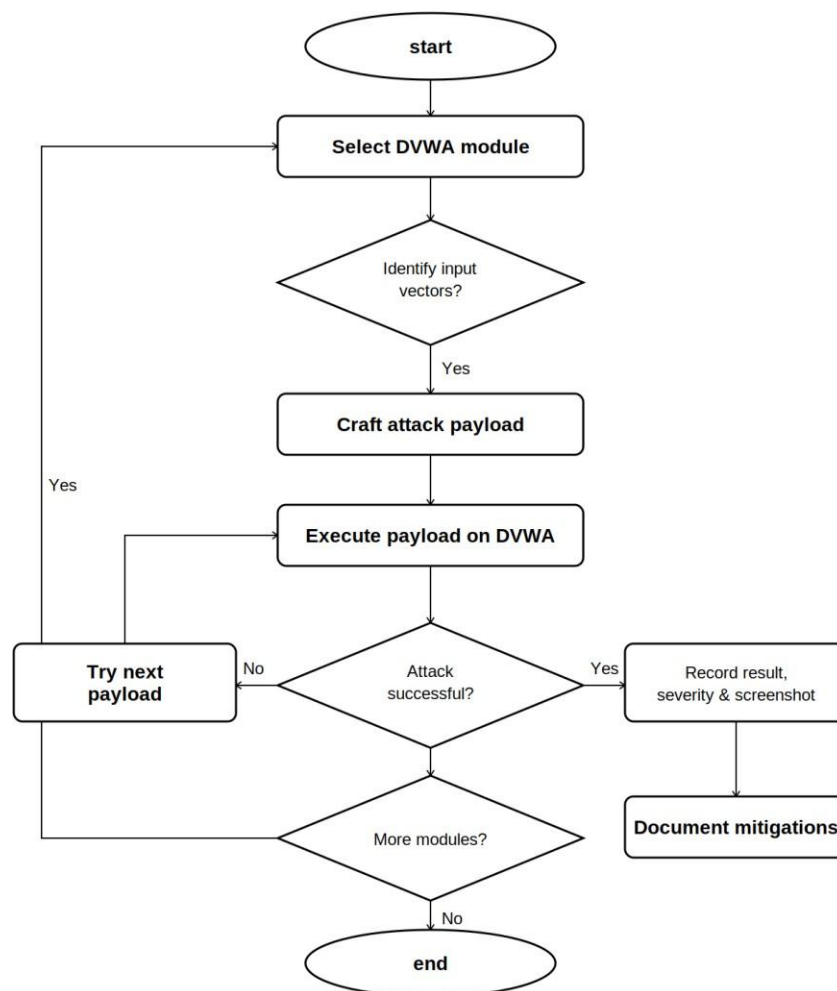
Phase 4: Exploitation

The identified vulnerabilities were tested using ethical hacking techniques. Appropriate payloads were applied to verify whether the vulnerabilities could be exploited successfully. All testing was performed only within the controlled laboratory environment.

Phase 5: Documentation and Analysis

The results of each attack were documented with screenshots, payloads, observations, severity levels, and recommended mitigation techniques.

Flowchart



Experimental Setup

4.1 Laboratory Environment

The penetration testing experiments were conducted in a controlled laboratory environment to ensure that no external systems were affected. All attacks were performed only on DVWA, an intentionally vulnerable web application designed for security education.

The laboratory environment consisted of:

Component	Description
Operating System	Kali Linux
Target Application	DVWA (Damn Vulnerable Web Application)
Web Server	Apache2
Database Server	MySQL/MariaDB
Programming Language	PHP
Browser	Mozilla Firefox
Security Level	Low

Tools Used

Several tools were used during the penetration testing process. Each tool played an important role in identifying and exploiting web application vulnerabilities.

5.1 Kali Linux

Kali Linux is a Debian-based Linux distribution specifically designed for penetration testing, digital forensics, and cybersecurity research. It provides hundreds of built-in security tools for vulnerability assessment and ethical hacking.

Purpose in this project:

- Running DVWA
- Executing penetration testing commands
- Managing the testing environment

5.2 DVWA (Damn Vulnerable Web Application)

DVWA is an intentionally vulnerable PHP/MySQL web application developed for security training and ethical hacking practice. It allows students and professionals to understand common web vulnerabilities in a safe environment.

Purpose in this project:

- Target application for testing
- Demonstration of web vulnerabilities
- Educational penetration testing

5.3 Burp Suite Community Edition

Burp Suite is one of the most widely used web application penetration testing tools. It works as an intercepting proxy, allowing testers to capture, inspect, and modify HTTP requests and responses.

In this project, Burp Suite was primarily used to perform the **Brute Force attack** by intercepting login requests and using the Intruder module to automate password testing.

Purpose in this project:

- Intercept HTTP requests
- Analyze request and response data
- Perform Brute Force testing
- Inspect application traffic

5.4 Mozilla Firefox

Mozilla Firefox was used as the primary browser to access DVWA and interact with the vulnerable modules during testing.

Purpose in this project:

- Accessing DVWA
- Executing attack payloads
- Observing results

6. About DVWA

Damn Vulnerable Web Application (DVWA) is an open-source PHP/MySQL web application intentionally designed with security vulnerabilities for educational purposes. It provides a safe environment where students and security professionals can learn web application penetration testing techniques without targeting real-world systems.

DVWA includes multiple vulnerable modules such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, File Inclusion, CSRF, File Upload, Brute Force Authentication, and others. These modules allow learners to understand how vulnerabilities occur, how they can be exploited, and how they should be mitigated.

For this project, the Low Security Level configuration was selected to demonstrate the exploitation of common web application vulnerabilities.

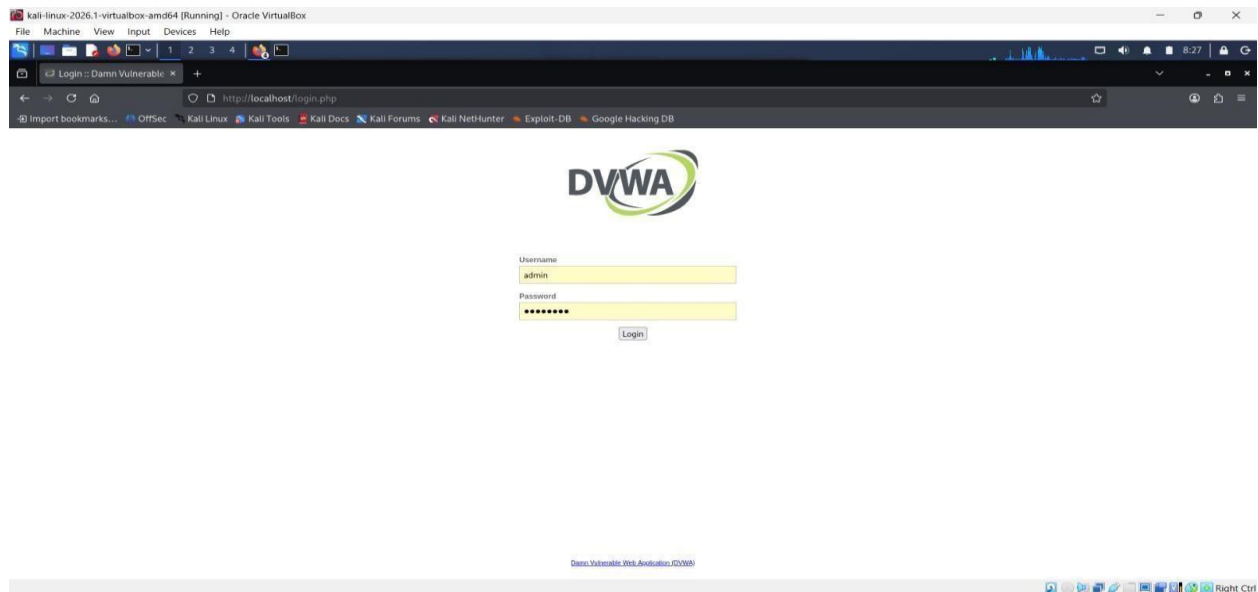


Figure 1: DVWA Lab Setup - Penetration Testing Environment

7. Vulnerability Assessment and Exploitation

7.1 SQL Injection (SQLi)

7.1.1 Introduction

SQL Injection (SQLi) is one of the most common and dangerous web application vulnerabilities. It occurs when user input is directly included in SQL queries without proper validation or parameterized statements. Attackers can manipulate database queries to bypass authentication, retrieve confidential information, modify records, or even delete database contents.

In DVWA, the SQL Injection module demonstrates how insecure input validation can allow attackers to execute malicious SQL commands against the application's database.

7.1.2 Objective

The objective of this test was to determine whether the application was vulnerable to SQL Injection attacks and whether unauthorized database information could be accessed through malicious SQL queries.

7.1.3 Testing Procedure

The SQL Injection module was opened from the DVWA dashboard.

The application requested a User ID as input.

Instead of entering a normal numeric value, SQL Injection payloads were inserted into the input field.

The application executed the SQL query without proper validation.

As a result, unauthorized database information was displayed.

Payload Used

`1' OR '1'='1`

`1' UNION SELECT user,password FROM users#`

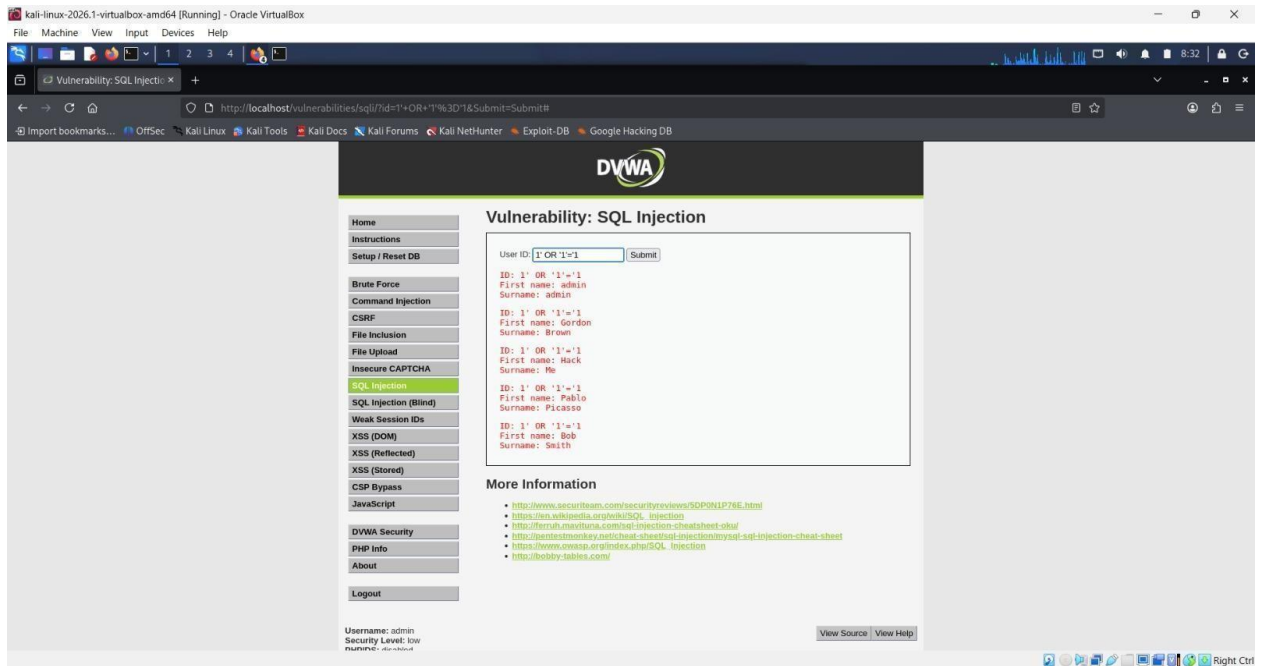


Figure 2: SQL Injection - Step 2

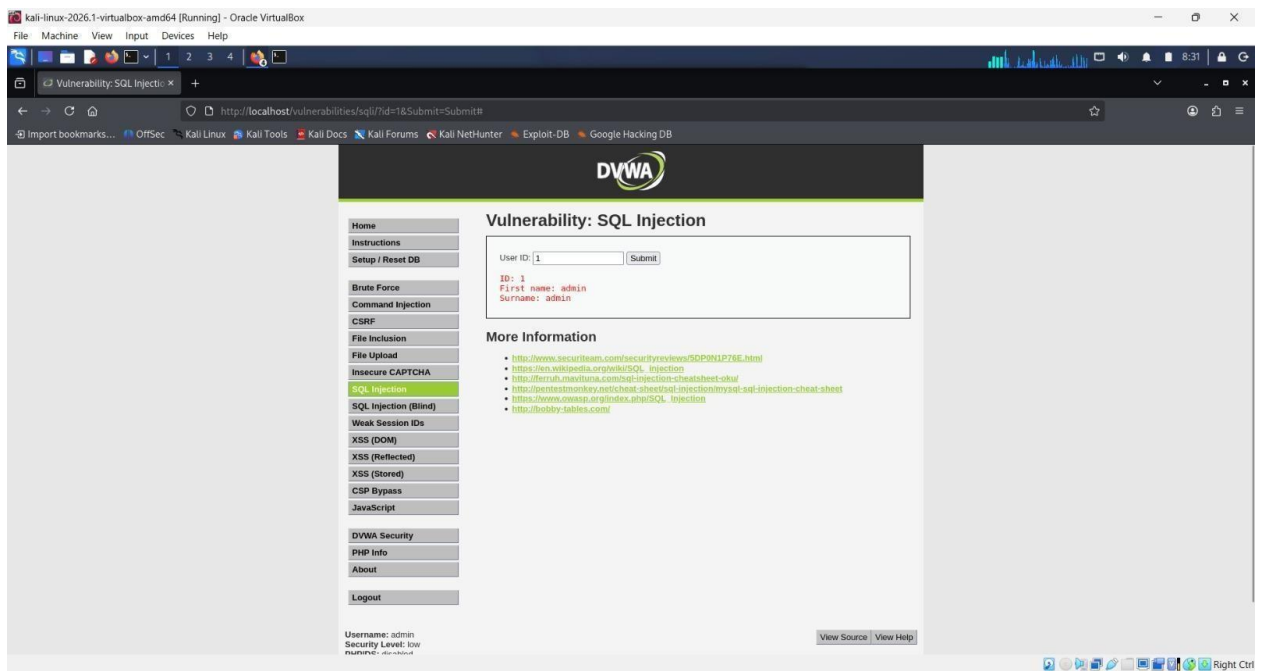


Figure 3: SQL Injection - Step 2

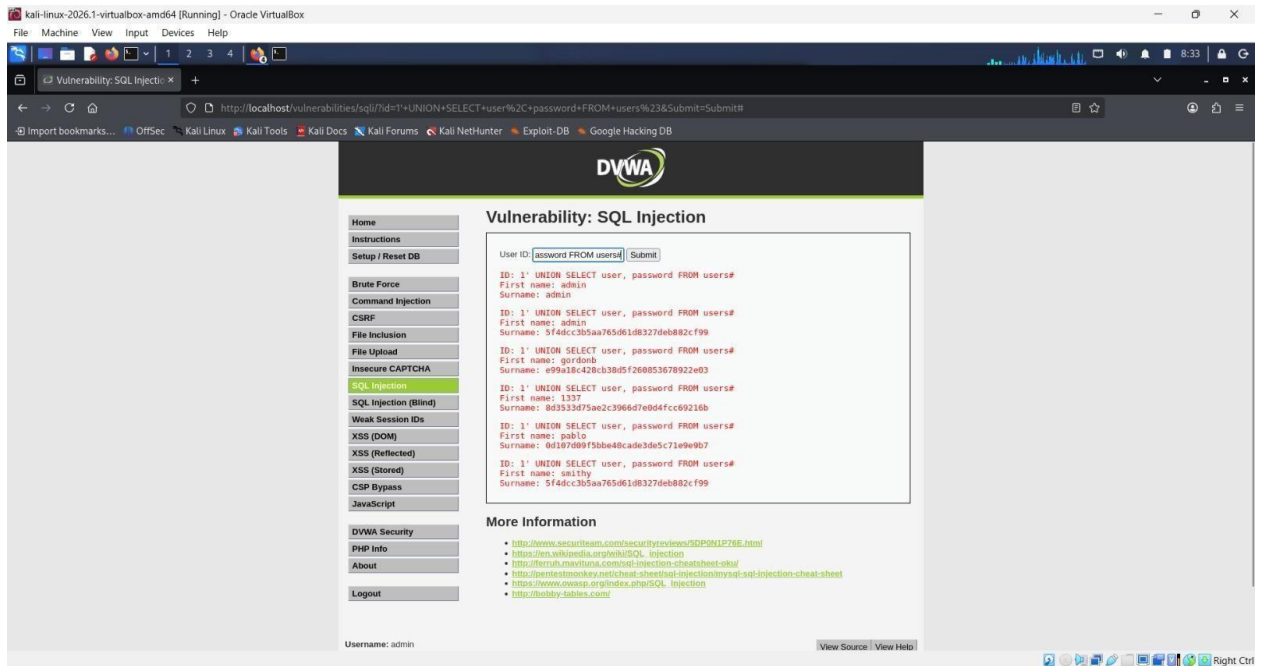


Figure 4: SQL Injection - Step 3

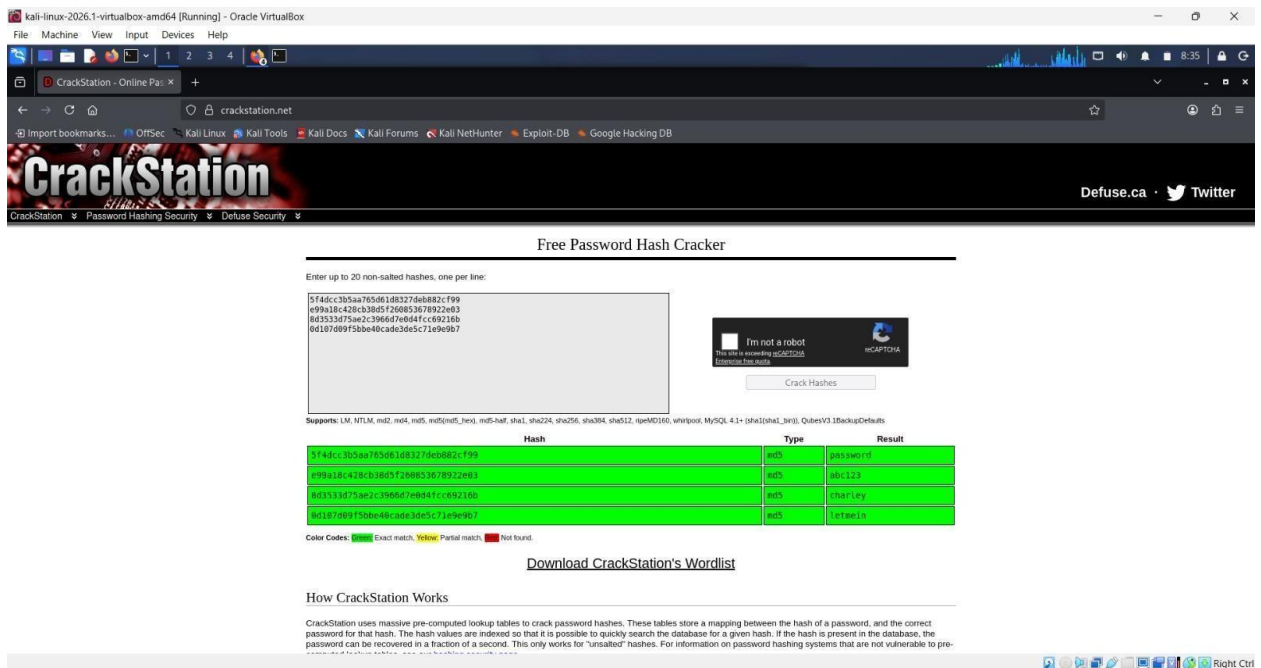


Figure 5: SQL Injection - Step 4

7.1.7 Result

The SQL Injection attack was successful.

The application returned user information stored in the database without proper authorization.

This confirmed that user input was directly executed as part of the SQL query.

7.1.8 Security Impact

The vulnerability allows attackers to:

- Read confidential database information
- Bypass authentication
- Extract usernames and passwords
- Modify or delete records
- Compromise database integrity

7.1.9 Risk Level

Severity: High

7.1.10 Mitigation

To prevent SQL Injection:

- Use Prepared Statements
- Use Parameterized Queries
- Validate user input
- Apply Least Privilege
- Use Web Application Firewall (WAF)

7.2 Blind SQL Injection

7.2.1 Introduction

Blind SQL Injection is a type of SQL Injection where the application does not directly display database errors or query results. Instead, attackers determine whether injected SQL statements are true or false by observing differences in the application's behavior.

Although no database information is directly shown, attackers can gradually extract sensitive information through repeated requests.

7.2.2 Objective

The objective was to verify whether the application responded differently to true and false SQL conditions, indicating the presence of Blind SQL Injection.

7.2.4 Testing Procedure

The Blind SQL Injection module was opened.

Boolean-based SQL payloads were entered.

The application responses were observed.

Differences between true and false conditions confirmed the vulnerability

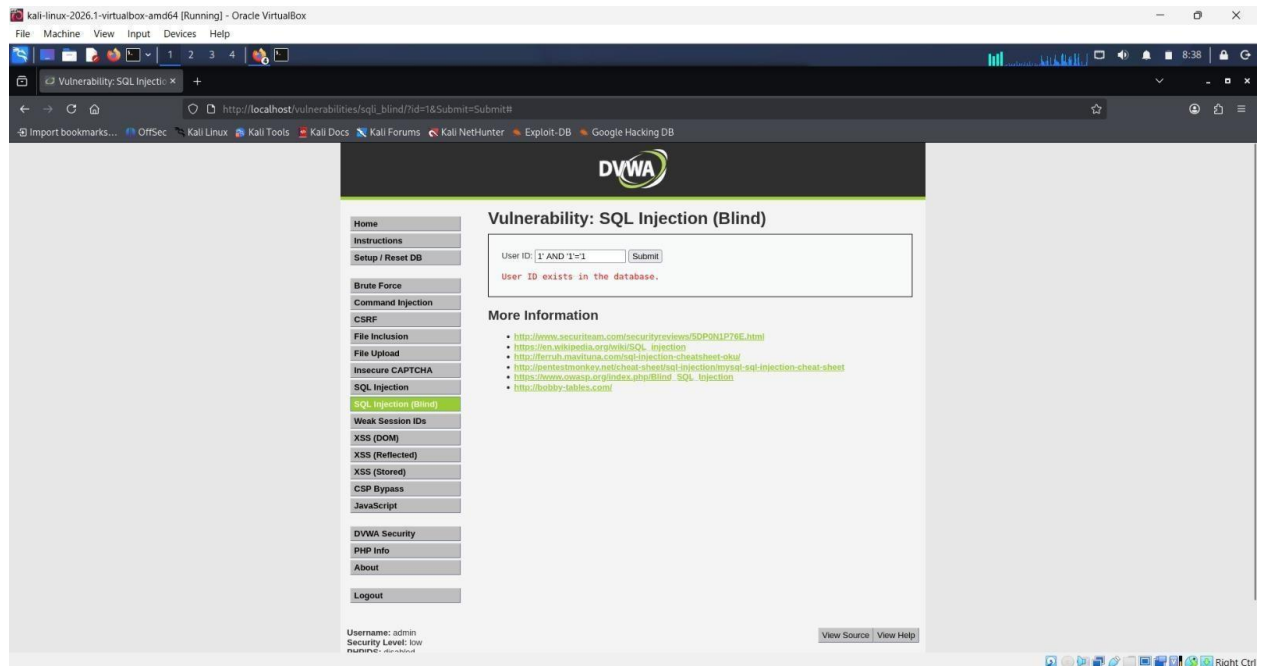


Figure 6: Blind SQL Injection - Step 1

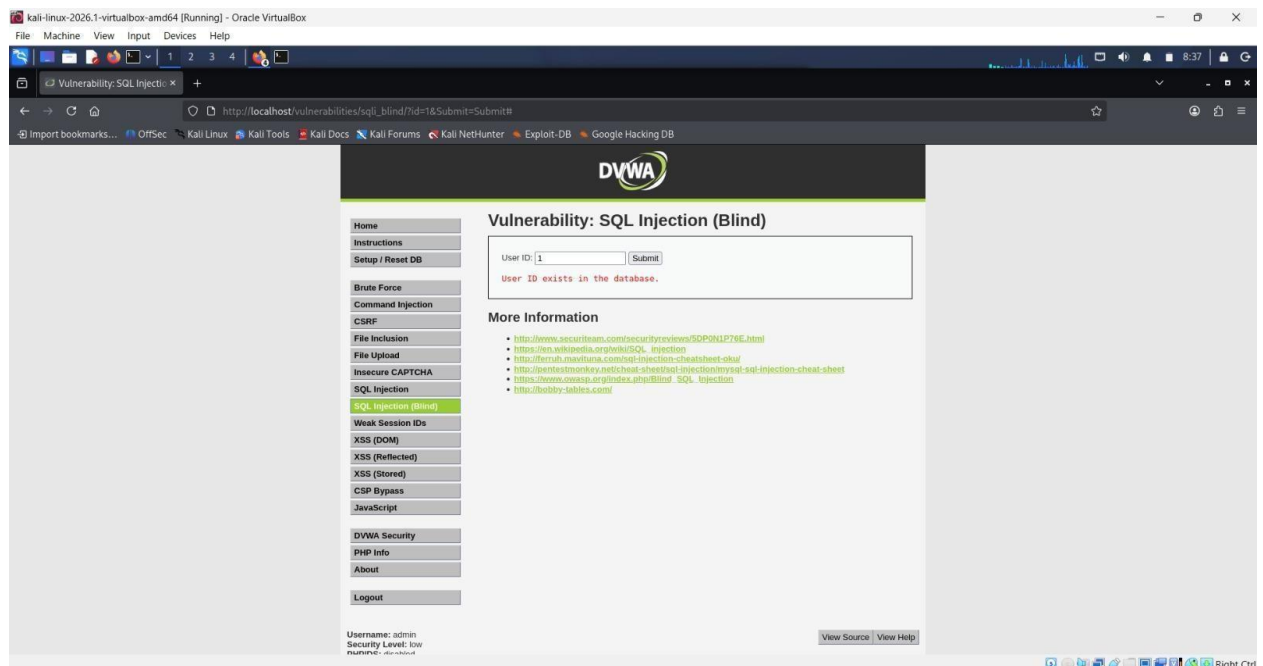


Figure 7: Blind SQL Injection - Step 2

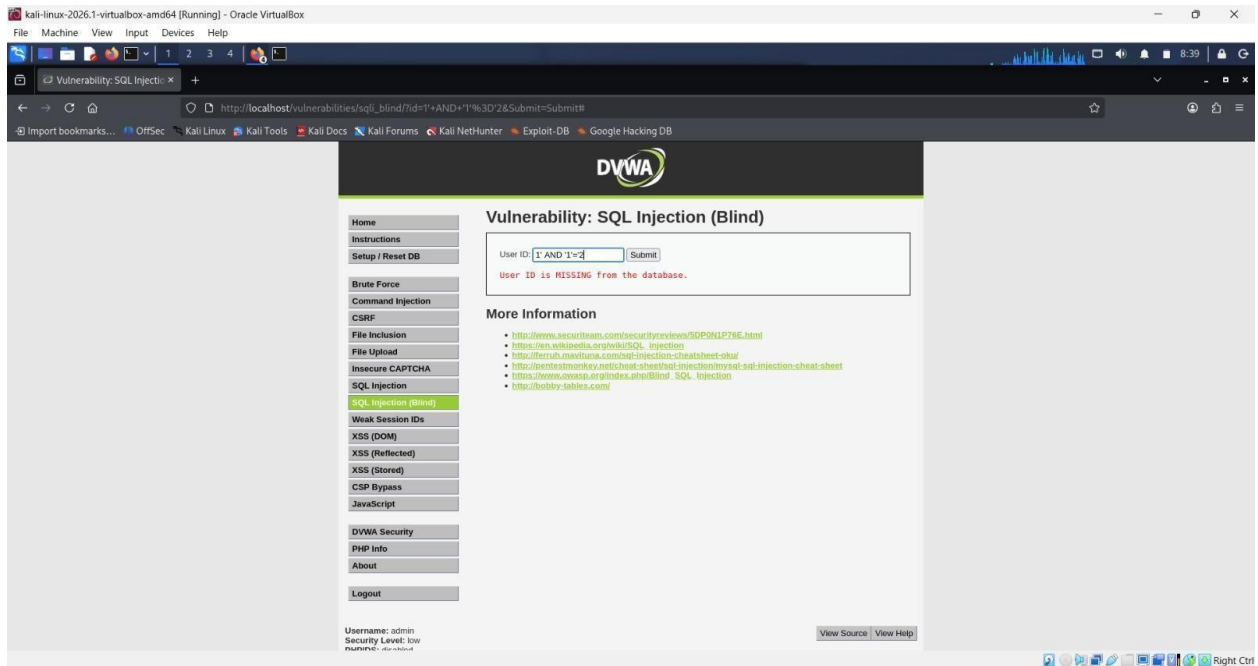


Figure 8: Blind SQL Injection - Step 3

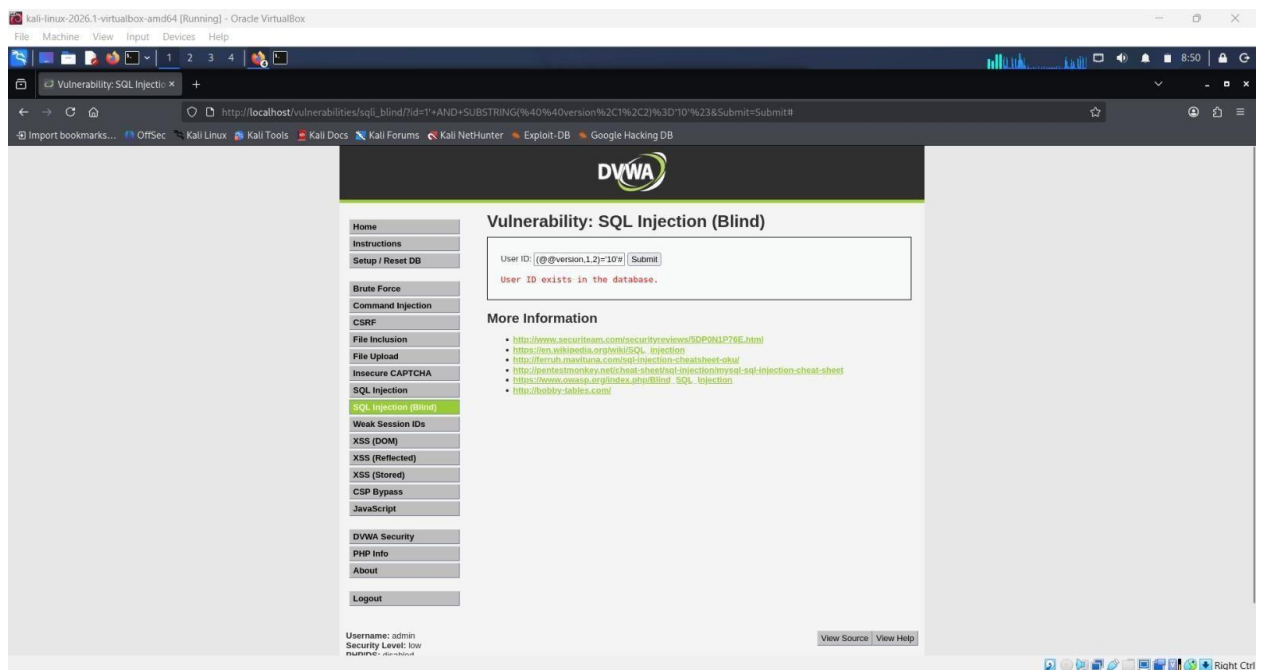


Figure 9: Blind SQL Injection - Step 4

7.2.7 Result

The application produced different responses for true and false conditions.

This behavior confirmed that the application was vulnerable to Blind SQL Injection.

7.2.8 Security Impact

Attackers may:

- Extract database information slowly
- Enumerate database tables
- Discover usernames
- Retrieve password hashes
- Perform database fingerprinting

7.2.9 Risk Level

Severity: High

7.2.10 Mitigation

Recommended security controls include:

- Prepared Statements
- Input Validation
- Stored Procedures
- Error Message Suppression
- Parameterized Queries

7.3 Brute Force Attack

7.3.1 Introduction

A Brute Force Attack is an authentication attack in which an attacker repeatedly tries different username and password combinations until the correct credentials are found. This attack is effective against weak passwords or applications that do not implement account lockout, CAPTCHA, or rate limiting.

In this project, the Brute Force vulnerability of DVWA was tested using Burp Suite Community Edition. The Burp Intruder module was used to automate multiple login attempts and identify valid credentials.

7.3.2 Objective

The objective of this test was to determine whether the login page was vulnerable to brute force attacks and to evaluate the effectiveness of the application's authentication mechanism.

7.3.4 Testing Procedure

1. DVWA was opened in Mozilla Firefox.
2. The Brute Force module was selected.
3. Burp Suite Proxy was enabled to intercept the login request.
4. The intercepted request was sent to Burp Intruder.
5. The password parameter was selected as the attack position.
6. A password list containing common passwords was loaded into Intruder.
7. The attack was started, and server responses were analyzed.
8. A successful login response indicated valid credentials.

7.3.6 Screenshot

```

kali@kali:~$ sudo docker pull vulnerabilities/web-dvwa
[sudo] password for kali:
Using default tag: latest
latest: Pulling from vulnerabilities/web-dvwa
Digest: sha256:dac281fe1356a86917bf042b079ade295f426da6a92b40e3b181f337daa7
Status: Image is up to date for vulnerabilities/web-dvwa:latest
kali@kali:~$ sudo docker run -d --name dvwa vulnerabilities/web-dvwa
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on end
point peaceful_goddall (8370a631802b0e0f0dc23c00b5cd483c24d3d40b7a38c674ce0a855e4e919f8d7): Bind for 0.0.0.0:80 failed: port is al
ready allocated
Run 'docker run --help' for more information
kali@kali:~$ curl -s -c /tmp/cookies.txt -d "username=admin&password=passwordLogin=Login" http://localhost/login.php
cat /tmp/cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

localhost FALSE / FALSE 0 security low
localhost FALSE / FALSE 0 PHPSESSID 6vdop69pap43s03bdjhk01f133

```

Figure 10: Brute Force Attack - Step 1

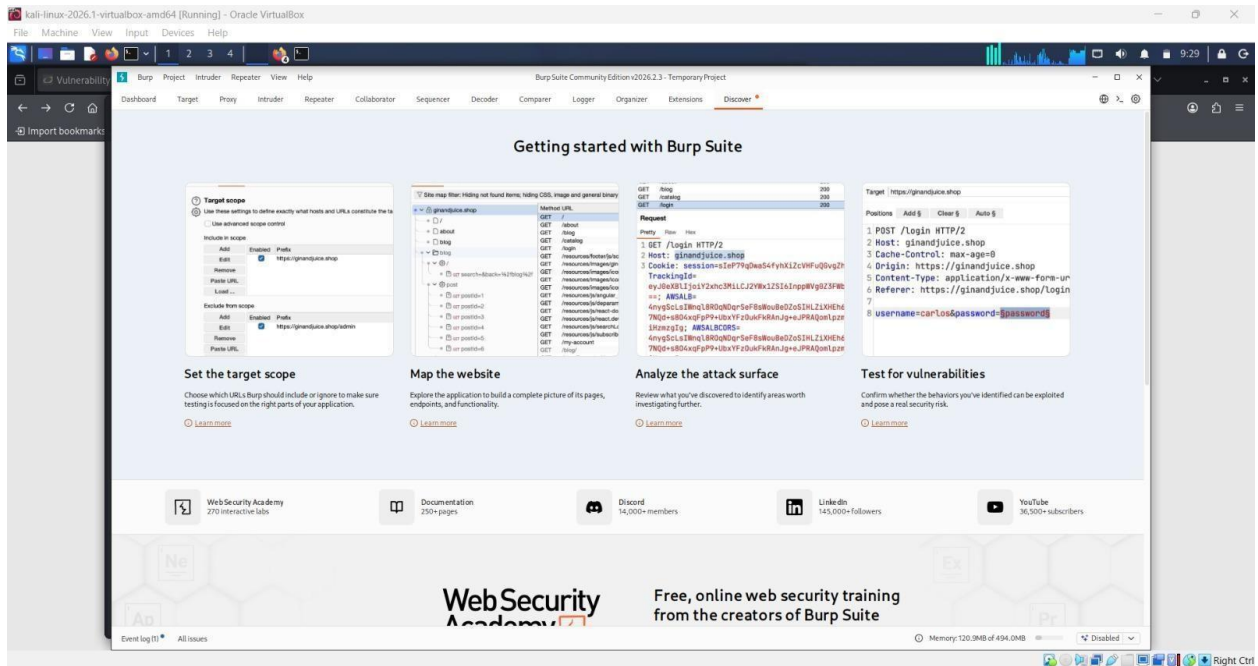


Figure 11: Brute Force Attack - Step 2

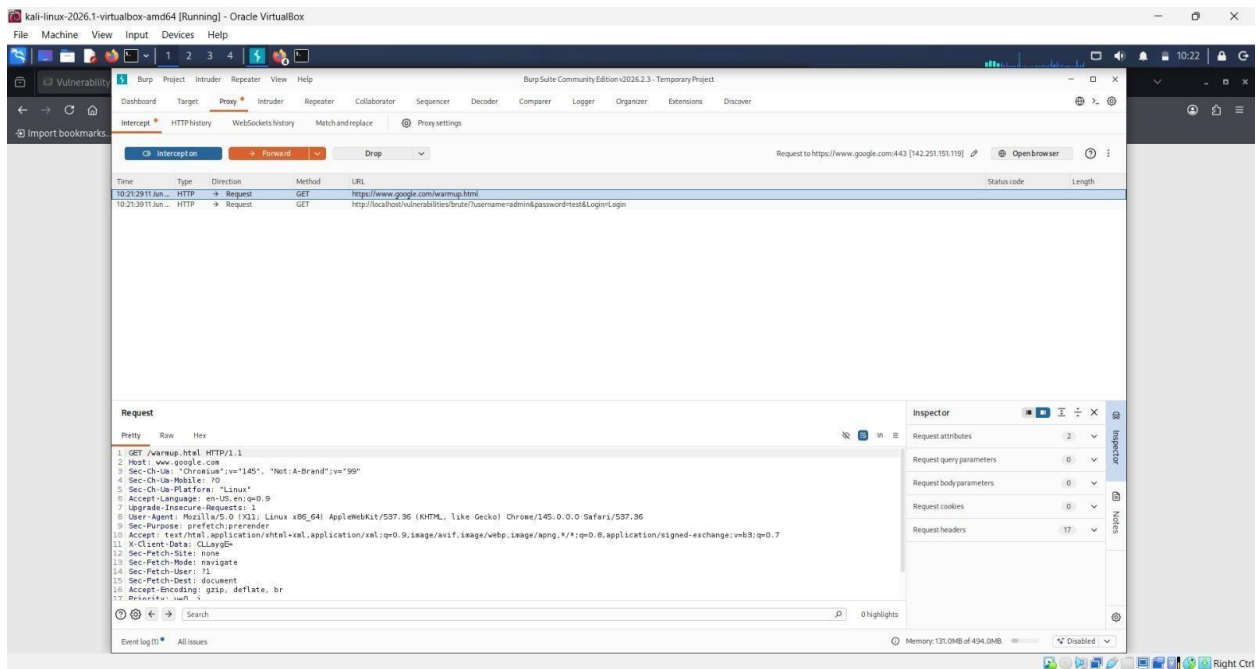


Figure 12: Brute Force Attack - Step 3

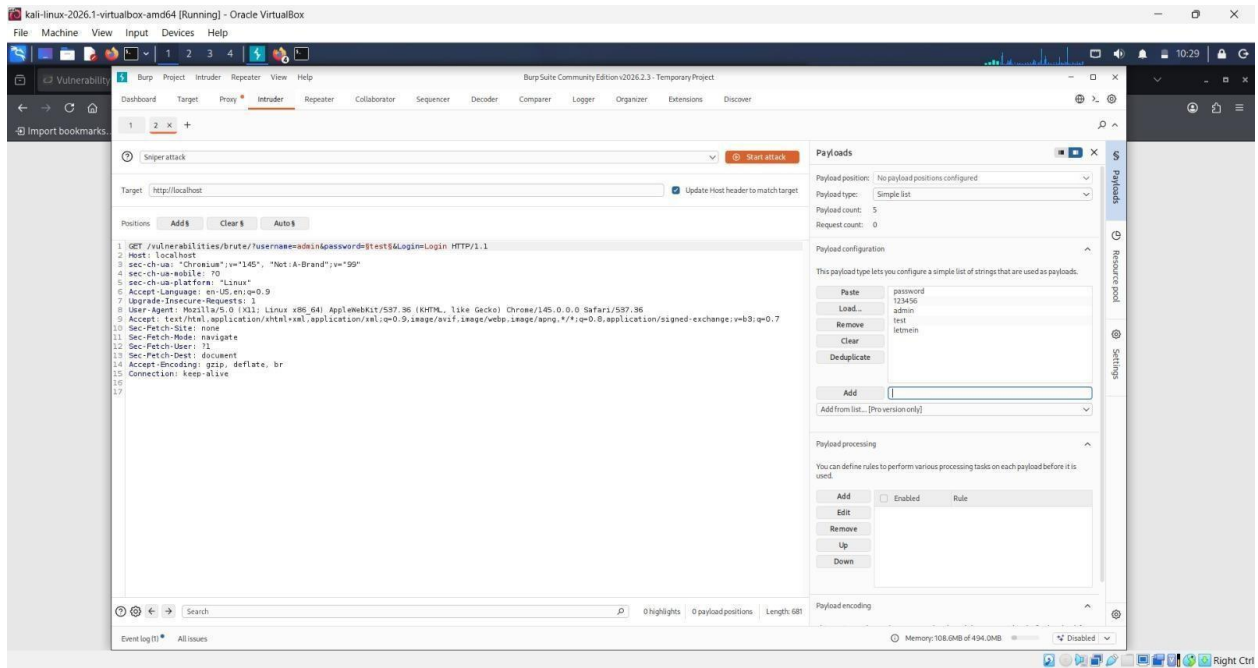


Figure 13: Brute Force Attack - Step 4

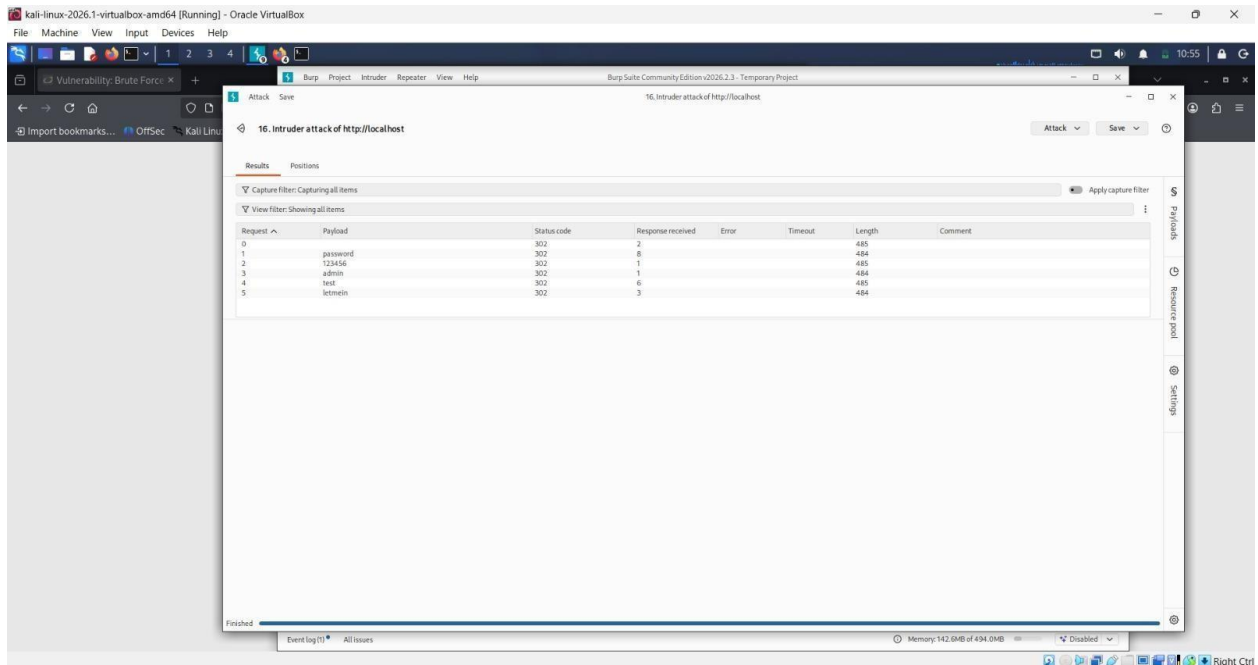


Figure 14: Brute Force Attack - Step 5

7.3.7 Result

The Brute Force attack was successful because the application did not implement account lockout or rate limiting at the Low security level. The correct credentials were identified after multiple login attempts using Burp Suite Intruder.

7.3.8 Security Impact

A successful Brute Force attack may allow attackers to:

- Gain unauthorized access to user accounts.
- Compromise administrator accounts.
- Access confidential information.
- Escalate privileges.
- Launch further attacks from authenticated sessions.

7.3.9 Risk Level

Severity: Medium

7.3.10 Mitigation

The following security measures are recommended:

- Implement strong password policies.
- Enable account lockout after repeated failed login attempts.
- Use CAPTCHA to prevent automated login attempts.
- Monitor and log suspicious authentication attempts. Enable Multi-Factor Authentication (MFA).

7.4 Command Injection

7.4.1 Introduction

Command Injection is a critical vulnerability that occurs when user input is passed directly to the operating system without proper validation. An attacker can inject additional operating system commands and execute them with the privileges of the web server.

In DVWA, the Command Injection module demonstrates how insecure input handling can lead to arbitrary command execution.

7.4.2 Objective

The objective of this assessment was to determine whether operating system commands could be executed by manipulating user input.

7.4.4 Testing Procedure

1. The Command Injection module was opened.
2. A valid IP address was entered.
3. A command separator (&&) was used to append an additional system command.
4. The request was submitted.
5. The output displayed both the ping result and the injected command output.



Figure 15: Command Injection - Step 1



Figure 16: Command Injection - Step 2

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript

DVWA Security

PHP Info

About

Logout

DVWA

Vulnerability: Command Injection

Ping a device

Enter an IP address:

Submit

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.212 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.383 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.138 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.223 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.138/0.237/0.383/0.092 ms
bin
boot
dev
etc
home
lib
lib64
main.sh
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
```

More Information

- <http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://www.owasp.org/index.php/Command_injection

Figure 17: Command Injection - Step 3

Page 23 of 41



Figure 18: Command Injection - Step 4

7.4.7 Result

The application executed the injected operating system command successfully. The output confirmed that arbitrary commands could be executed through the vulnerable input field.

7.4.8 Security Impact

Command Injection can lead to:

- Remote Code Execution (RCE).
- Unauthorized access to server resources.
- Reading sensitive system files.
- Installing malicious software.
- Complete server compromise.

7.4.9 Risk Level

Severity: Critical

7.4.10 Mitigation

The following countermeasures are recommended:

- Avoid passing user input directly to system commands.
- Validate and sanitize all user inputs.
- Use allowlists for accepted input values.
- Execute commands with the least required privileges.
- Disable unnecessary system command execution.
- Implement server-side security controls.

7.5 Cross-Site Request Forgery (CSRF)

7.5.1 Introduction

Cross-Site Request Forgery (CSRF) is a web security vulnerability that forces an authenticated user to perform unwanted actions on a web application without their knowledge or consent. Since the victim is already logged into the application, the browser automatically includes valid session cookies with the malicious request. As a result, the server processes the request as if it were sent by the legitimate user.

In DVWA, the CSRF module demonstrates how the absence of anti-CSRF protections allows attackers to manipulate user actions by crafting unauthorized requests

7.5.2 Objective

The objective of this experiment was to determine whether the application was vulnerable to Cross-Site Request Forgery attacks and whether sensitive user actions, such as changing an account password, could be performed without proper verification.

7.5.4 Testing Procedure

1. Login to DVWA.
2. Navigate to the **CSRF** module.
3. Enter a new password and confirm it.
4. Observe the request URL generated by the application.
5. Copy the request into a separate HTML page.
6. Open the HTML page while remaining logged into DVWA.
7. The password is changed automatically without additional verification.



Figure 19: Cross-Site Request Forgery (CSRF) - Step 1

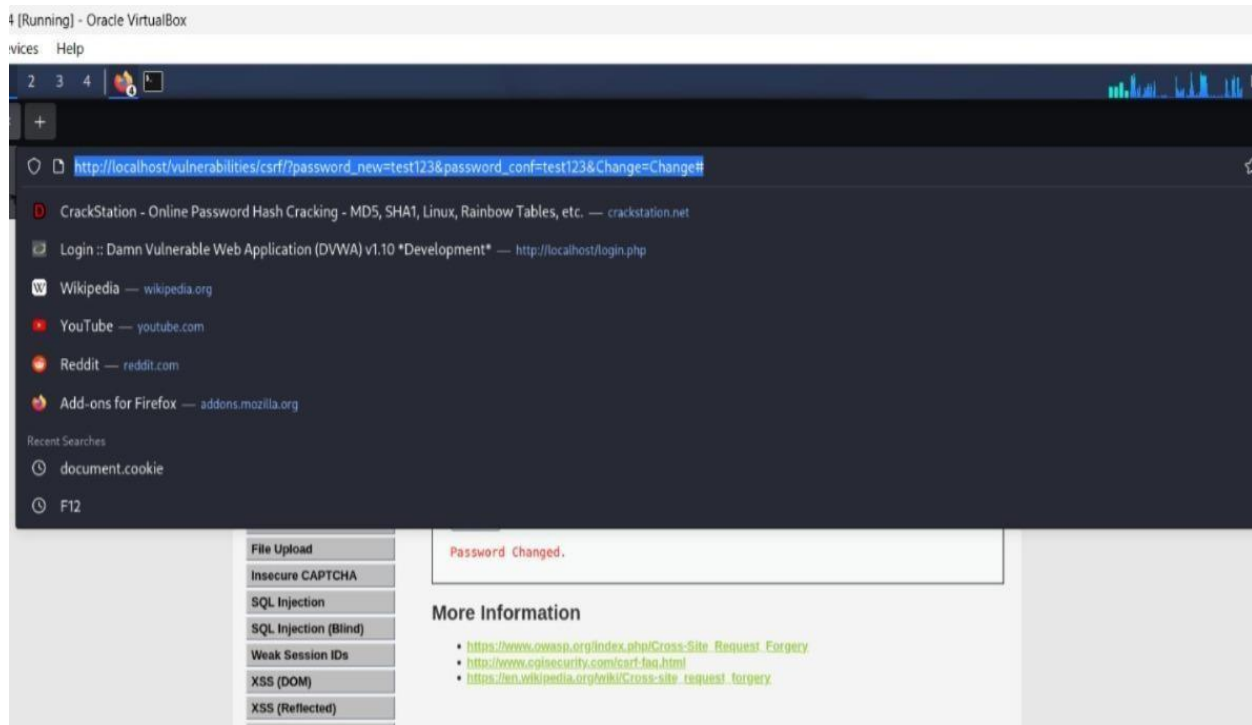


Figure 20: Cross-Site Request Forgery (CSRF) - Step 2

7.5.7 Result

The attack was successful. The application accepted the forged request and changed the user's password without verifying the legitimacy of the request. This confirmed that the application lacked CSRF protection mechanisms.

7.5.8 Security Impact

A successful CSRF attack may allow attackers to:

- Change user passwords.
- Modify account settings.
- Transfer sensitive information.
- Perform unauthorized transactions.
- Execute actions on behalf of authenticated users.

7.5.9 Risk Level

Severity: Medium

7.5.10 Mitigation

The following security measures should be implemented:

- Use Anti-CSRF Tokens.
- Verify the HTTP Referer and Origin headers.
- Implement SameSite cookies.
- Require user confirmation for sensitive actions.
- Use Multi-Factor Authentication (MFA)

7.6 File Inclusion

7.6.1 Introduction

File Inclusion is a vulnerability that allows attackers to include unintended files in a web application. Improper validation of user-supplied file paths may enable attackers to access sensitive local files (Local File Inclusion - LFI) or, in some configurations, execute remote files (Remote File Inclusion - RFI).

In DVWA, the File Inclusion module demonstrates how insecure file handling can expose sensitive information stored on the server.

7.6.2 Objective

The objective of this assessment was to determine whether the application allowed unauthorized access to local system files through manipulated file path parameters.

7.6.4 Testing Procedure

1. Open the **File Inclusion** module.
2. Observe the page parameter in the URL.
3. Modify the parameter by inserting directory traversal sequences.
4. Request a sensitive system file.
5. **Analyze the server response.**



Figure 21: File Inclusion - Step 1

```
(kali@kali)-[~]
$ sudo docker exec -it $(sudo docker ps -q) cat /etc/passwd
[sudo] password for kali:
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/bin/false
mysql:x:101:101:MySQL Server,,,:/nonexistent:/bin/false

(kali@kali)-[~]
$
```

Figure 22: File Inclusion - Step 2

7.6.7 Result

The application successfully displayed the contents of the system file. This confirmed that user-controlled file paths were not properly validated and that the application was vulnerable to Local File Inclusion.

7.6.8 Security Impact

This vulnerability may allow attackers to:

- Read sensitive system files.
- Obtain user account information.
- Reveal server configuration details.
- Assist in privilege escalation.
- Prepare for further attacks.

7.6.9 Risk Level

Severity: High

7.6.10 Mitigation

Recommended countermeasures include:

- Validate and sanitize file path inputs.
- Use an allowlist of permitted files.
- Prevent directory traversal.
- Disable Remote File Inclusion where unnecessary.
- Restrict file system permissions.

7.7 File Upload Vulnerability

7.7.1 Introduction

A File Upload Vulnerability occurs when a web application allows users to upload files without proper validation. If the application does not verify the file type, extension, content, or storage location, an attacker may upload malicious files such as PHP web shells. These files can be executed on the server, potentially leading to Remote Code Execution (RCE) and full server compromise.

In DVWA, the File Upload module demonstrates how insecure upload functionality can be exploited by uploading executable files.

7.7.2 Objective

The objective of this assessment was to determine whether the application accepted malicious file uploads and whether the uploaded file could be executed from the server.

7.7.4 Testing Procedure

1. Open the File Upload module in DVWA.
2. Create or select a test PHP file.
3. Click Browse and choose the file.
4. Click Upload.
5. Verify whether the application accepts the file.
6. Access the uploaded file through the provided URL.
7. Observe whether the file executes successfully.



Figure 23: File Upload Vulnerability - Step 1



Figure 24: File Upload Vulnerability - Step 2

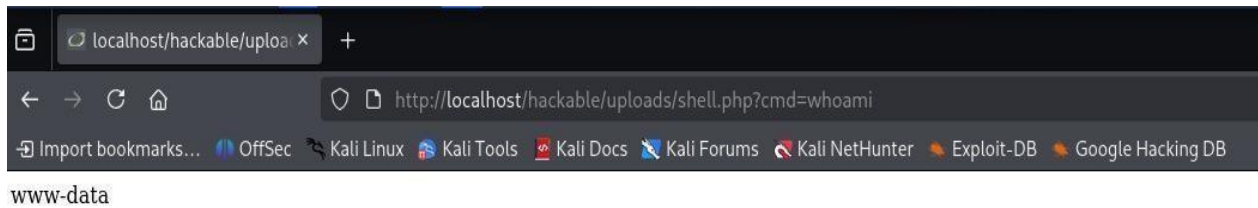


Figure 25: File Upload Vulnerability - Step 3

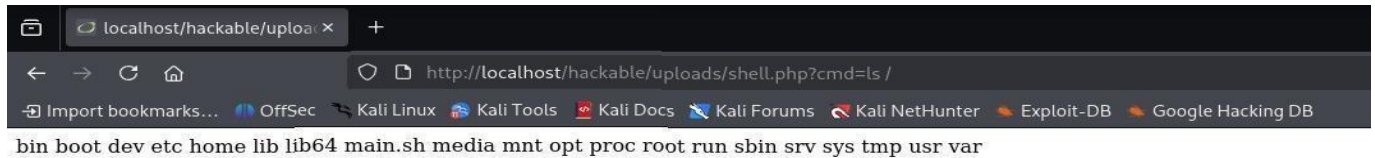


Figure 26: File Upload Vulnerability - Step 4

7.7.7 Result

The application accepted the uploaded PHP file without proper validation. The uploaded file was accessible through the browser and executed successfully, confirming the presence of an insecure file upload vulnerability.

7.7.8 Security Impact

A successful file upload vulnerability may allow attackers to:

- Upload executable scripts.
- Execute arbitrary server-side code.
- Access sensitive files.
- Modify web application content.
- Completely compromise the web server.

7.7.9 Risk Level

Severity: Critical

7.7.10 Mitigation

To prevent file upload vulnerabilities:

- Allow only required file types using an allowlist.
- Verify MIME type and file extension.
- Rename uploaded files.
- Store uploaded files outside the web root.
- Scan uploaded files with antivirus software.
- Restrict execution permissions for uploaded files.

7.8 DOM-Based Cross-Site Scripting (DOM XSS)

7.8.1 Introduction

DOM-Based Cross-Site Scripting (DOM XSS) is a client-side vulnerability where malicious input is processed by JavaScript in the browser rather than by the web server. The vulnerability occurs when JavaScript reads user-controlled data from the URL or another source and writes it into the page without proper sanitization.

Unlike Stored XSS or Reflected XSS, the malicious payload does not need to be processed by the server. The attack is executed entirely within the victim's browser.

7.8.2 Objective

The objective of this experiment was to determine whether JavaScript code supplied through the URL could be executed in the browser due to insecure DOM manipulation.

7.8.4 Testing Procedure

1. Open the DOM XSS module.
2. Modify the URL parameter used by the page.
3. Insert a JavaScript payload into the parameter.
4. Load the modified URL.
5. Observe whether the browser executes the injected script.

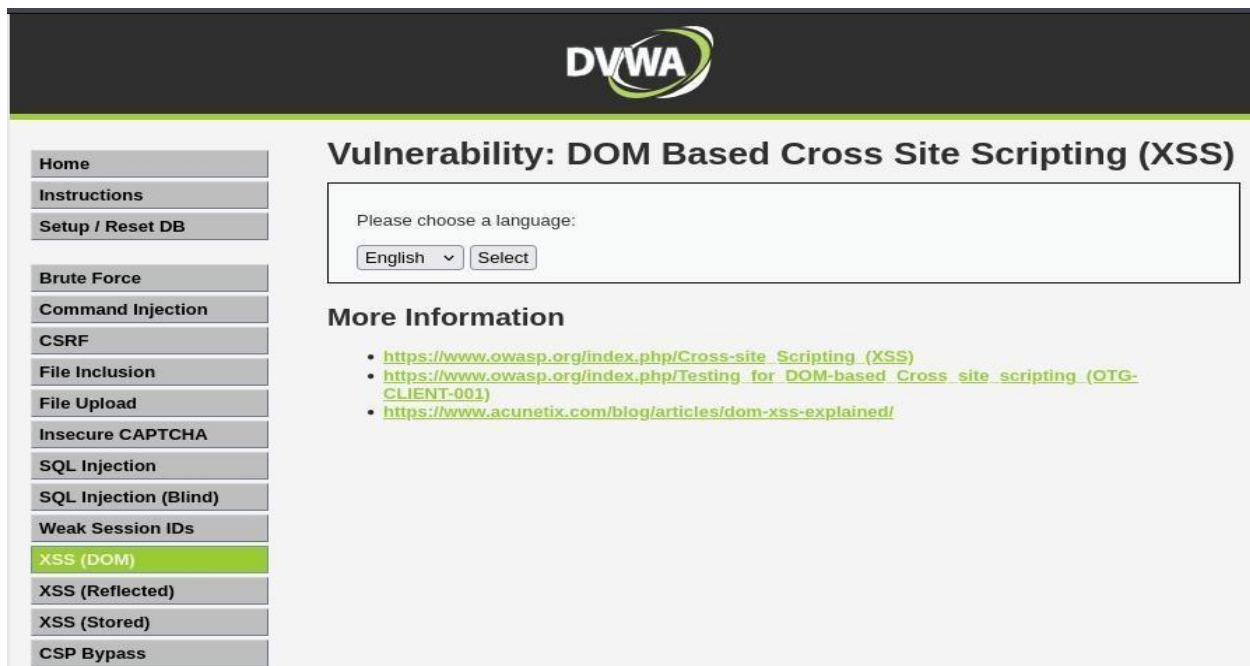


Figure 27: DOM-Based Cross-Site Scripting (XSS) - Step 1

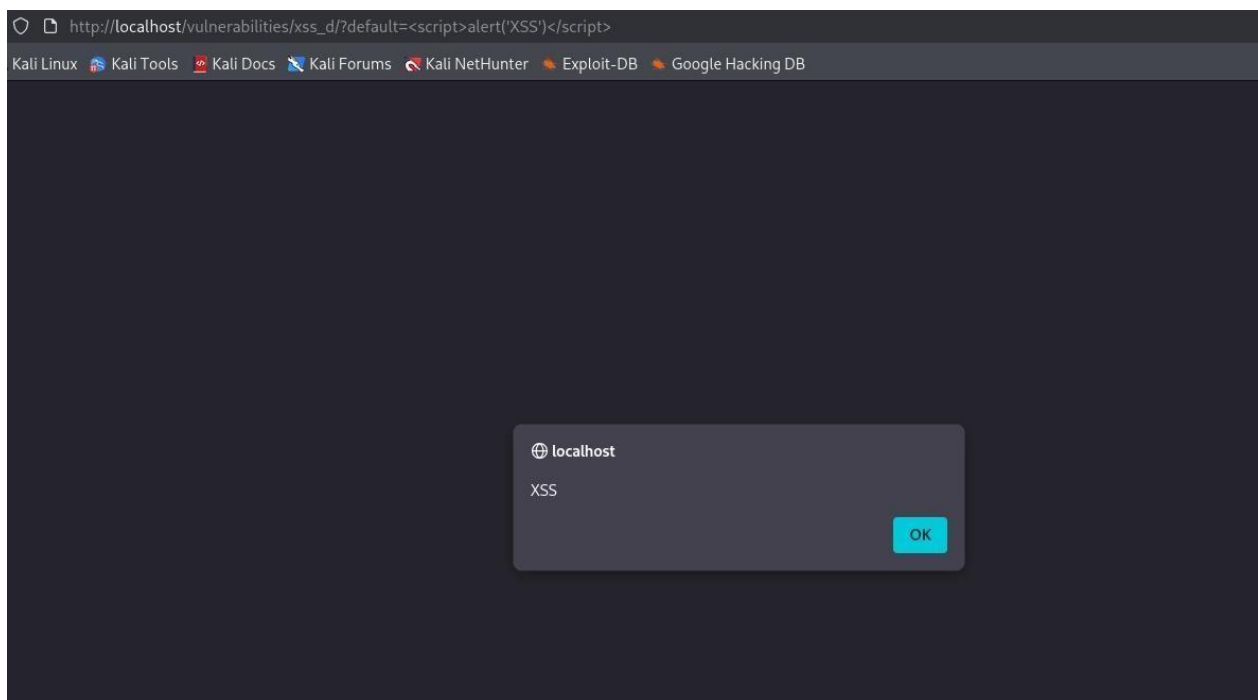


Figure 28: DOM-Based Cross-Site Scripting (XSS) - Step 2

7.8.8 Security Impact

A successful DOM XSS attack may allow attackers to:

- Execute arbitrary JavaScript in the victim's browser.

- Steal session cookies.
- Redirect users to malicious websites.
- Modify webpage content.
- Perform phishing attacks.

7.8.9 Risk Level

Severity: Medium

7.8.10 Mitigation

Recommended security measures include:

- Sanitize all client-side inputs.
- Avoid writing untrusted data directly into the DOM.
- Use safe DOM APIs such as `textContent` instead of `innerHTML`.
- Implement Content Security Policy (CSP).
- Validate and encode user input before processing.

7.9 Reflected Cross-Site Scripting (Reflected XSS)

7.9.1 Introduction

Reflected Cross-Site Scripting (Reflected XSS) is a vulnerability where malicious JavaScript is immediately reflected from the server in the HTTP response. The script is not stored on the server; instead, it is delivered through a crafted URL or form input. When a victim clicks the malicious link, the browser executes the injected script.

7.9.2 Objective

The objective of this test was to verify whether user-supplied input was reflected back into the webpage without proper output encoding.

7.9.3 Testing Procedure

1. Open the **Reflected XSS** module.
2. Enter a JavaScript payload in the input field.
3. Submit the request.
4. Observe whether the script executes in the browser.

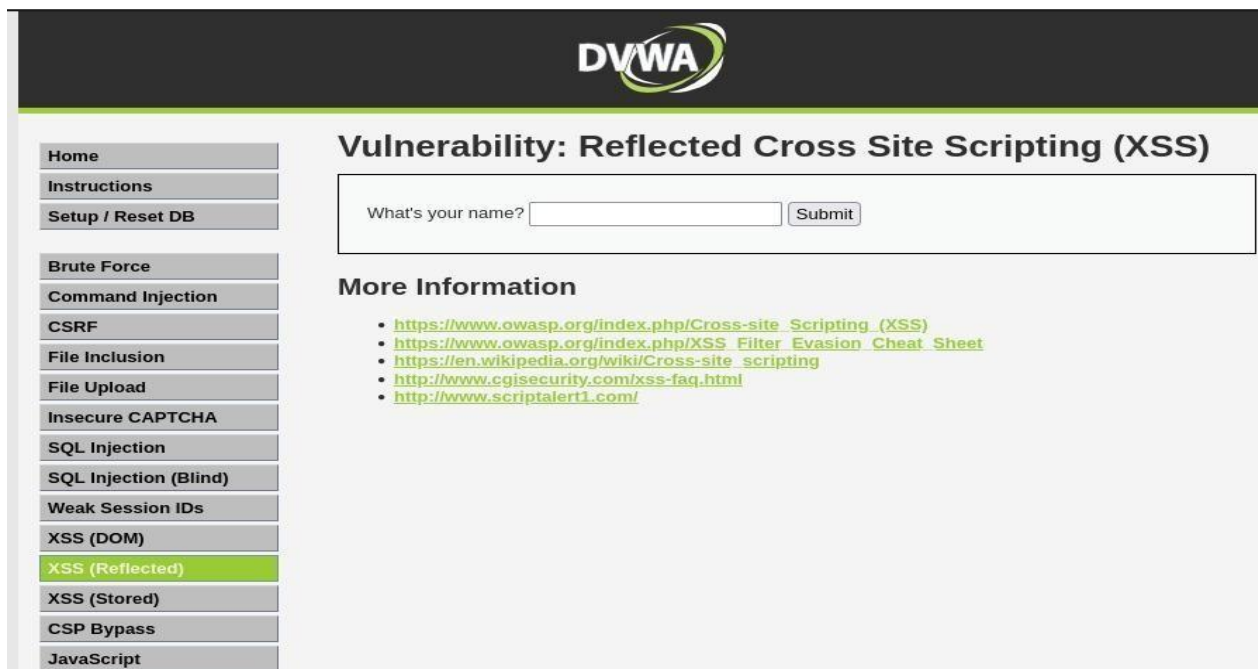


Figure 29: Reflected Cross-Site Scripting (XSS) - Step 1

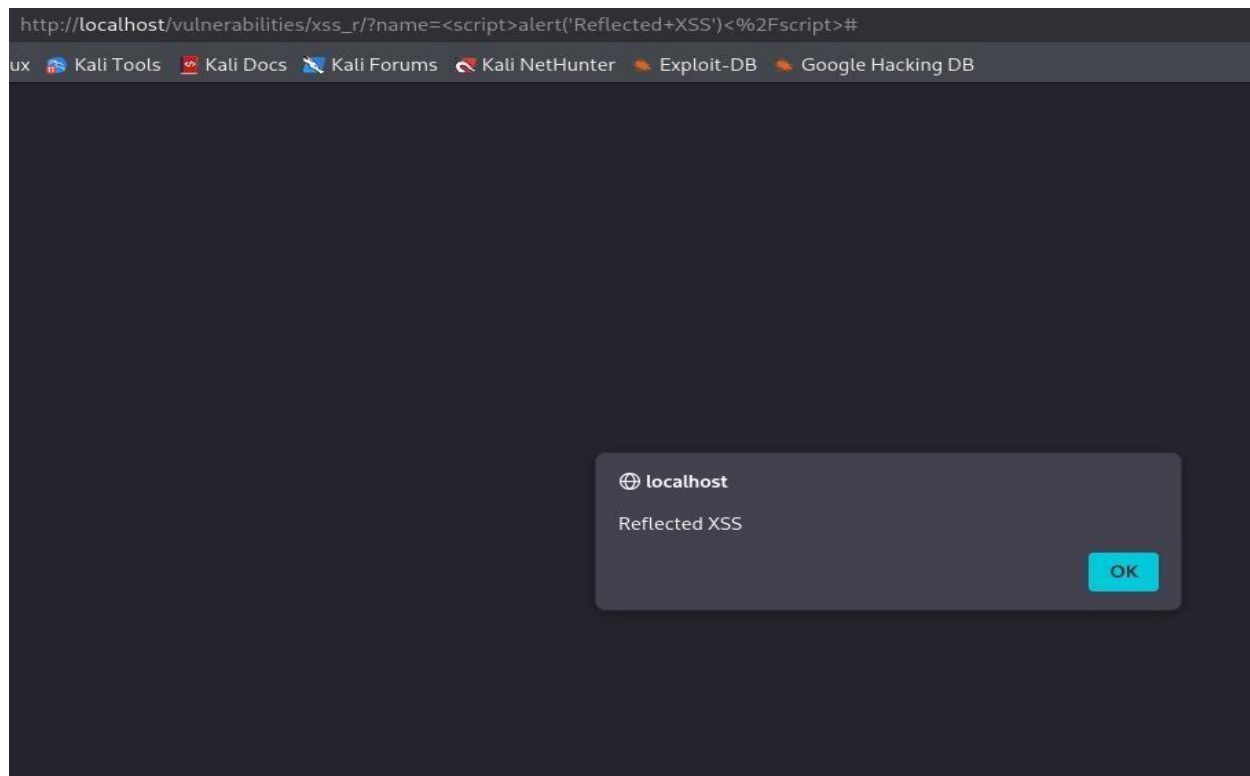


Figure 30: Reflected Cross-Site Scripting (XSS) - Step 2

7.9.6 Result

The browser executed the injected JavaScript successfully, confirming that the application reflected user input without proper output encoding.

7.9.7 Security Impact

- Session hijacking
- Cookie theft
- Credential theft
- Phishing attacks
- Unauthorized actions

7.9.8 Risk Level

Severity: Medium

7.9.9 Mitigation

- Encode output before displaying it.
- Validate all user input.
- Use Content Security Policy (CSP).

7.10 Stored Cross-Site Scripting (Stored XSS)

7.10.1 Introduction

Stored Cross-Site Scripting (Stored XSS) occurs when malicious JavaScript is permanently stored by the application, such as in a database, message board, or comment section. Every user who visits the affected page automatically executes the malicious script.

7.10.2 Objective

The objective was to determine whether user-supplied JavaScript could be permanently stored and executed whenever the page was viewed.

7.10.3 Testing Procedure

1. Open the Stored XSS module.
2. Enter a message containing JavaScript code.
3. Submit the form.
4. Reload the page.
5. Observe whether the script executes automatically.

DVWA

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Name: test
Message: This is a test comment.

More Information

- [https://www.owasp.org/index.php/Cross-site_Scripting_\(XSS\)](https://www.owasp.org/index.php/Cross-site_Scripting_(XSS))
- https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet
- https://en.wikipedia.org/wiki/Cross-site_scripting
- <http://www.cgisecurity.com/xss-faq.html>
- <http://www.scriptalert1.com/>

Figure 31: Stored Cross-Site Scripting (XSS) - Step 1

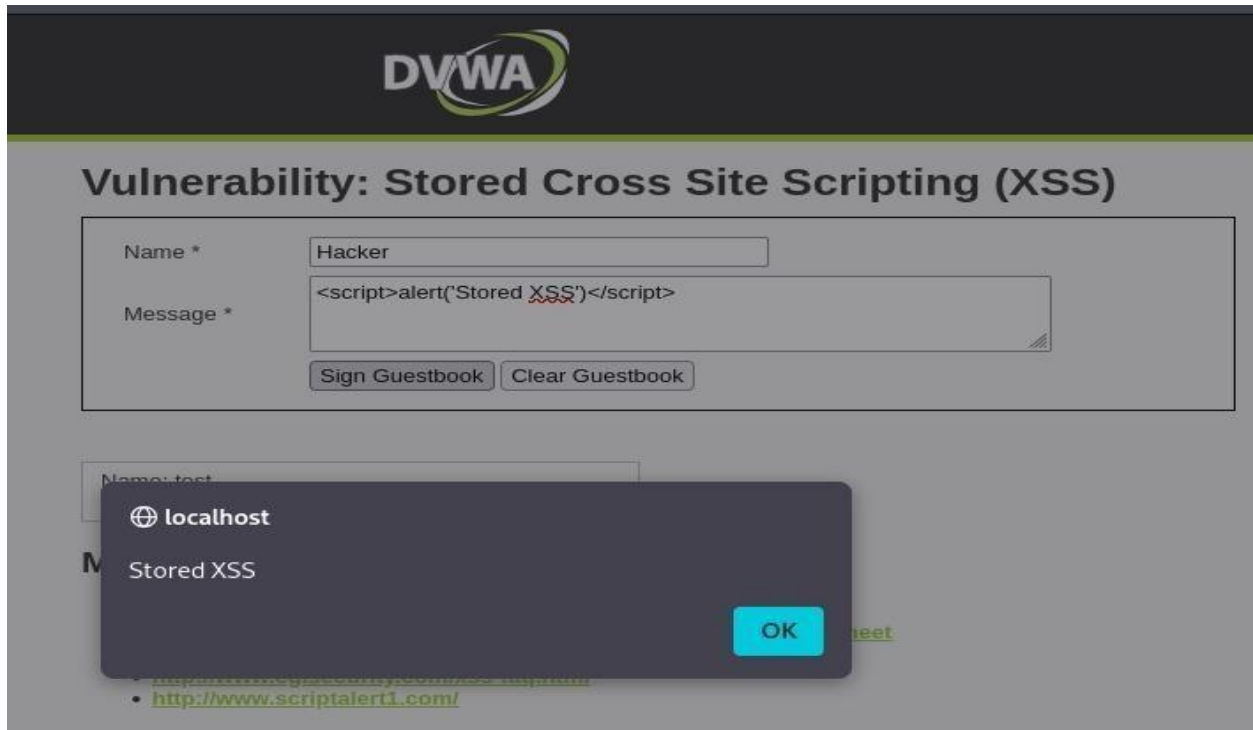


Figure 32: Stored Cross-Site Scripting (XSS) - Step 2

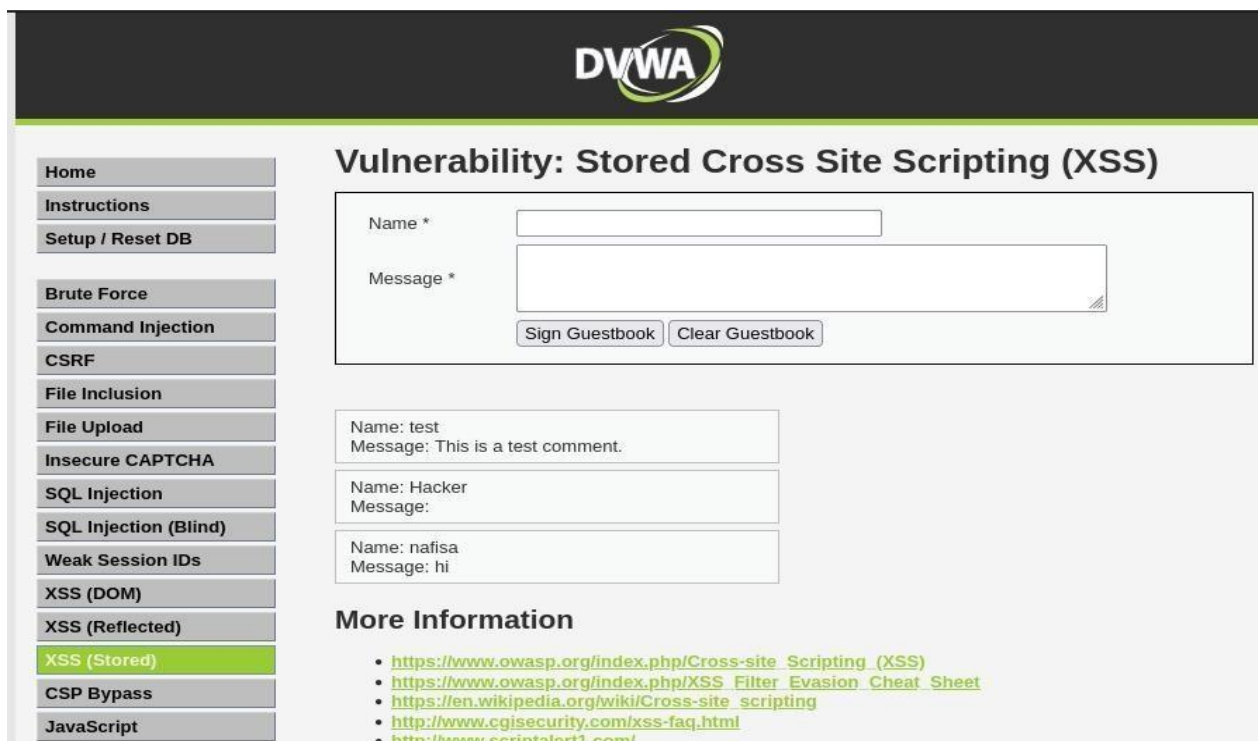


Figure 33: Stored Cross-Site Scripting (XSS) - Step 3

7.10.6 Result

The JavaScript payload was stored successfully and executed every time the page was loaded, confirming the Stored XSS vulnerability.

7.10.7 Security Impact

- Persistent session hijacking
- Cookie theft
- Defacement
- Phishing attacks
- Malware distribution

7.10.8 Risk Level

Severity: High

7.10.9 Mitigation

- Encode output before rendering.
- Validate and sanitize user input.
- Use Content Security Policy.
- Filter HTML and JavaScript content.
- Store only safe user data.

Results and Discussion

The penetration testing assessment demonstrated that the DVWA application at the Low Security Level contained multiple critical vulnerabilities. All ten selected vulnerabilities were successfully identified and exploited in a controlled laboratory environment.

The successful exploitation confirmed that the application lacked proper input validation, authentication controls, secure session management, output encoding, and secure file handling mechanisms. Among the tested vulnerabilities, Command Injection **and** File Upload posed the highest risk because they could potentially lead to remote code execution and complete server compromise.

SQL Injection and Blind SQL Injection exposed sensitive database information due to insecure database query construction. The Brute Force module showed weak authentication controls, while CSRF demonstrated the absence of request validation. DOM XSS, Reflected XSS, and Stored XSS highlighted insufficient output encoding and client-side security controls.

Overall, the experiment emphasized the importance of secure coding practices and regular security assessments during software development

Conclusion

This project successfully demonstrated a complete web application penetration testing assessment using Damn Vulnerable Web Application (DVWA). Ten common web application vulnerabilities were identified, exploited, analyzed, and documented in a secure laboratory environment.

The assessment highlighted how insecure coding practices can expose web applications to severe security threats. The results emphasize the importance of implementing secure authentication, proper input validation, secure file handling, output encoding, and regular penetration testing.

The practical experience gained through this project improved our understanding of ethical hacking methodologies, web application security, and vulnerability assessment techniques. These findings can help developers and security professionals design more secure web applications and reduce the risk of cyberattacks.